```
Ext.define('Hammerhead.grid.receiving.ReceivingLineItemGrid', {
    extend: 'Hammerhead.grid.CellField',
    alias: 'widget.receivinglineitemgrid',
    invalidReceivingValidationMsg: 'Rec Qty for at least one line item or checking the
 No Line Item Receiving box is required',
    stateId: 'receivingLineItemGrid',
    forceFit: false,
    requires: ['Hammerhead.view.customcontrols.LookupCombo',
               'Hammerhead.window.receiving.BoxItems',
               'Hammerhead.window.receiving.AssetItems',
               'Hammerhead.window.receiving.AssetReceivedItems',
               'Hammerhead.store.receiving.ReceivingLineItems',
               'Hammerhead.store.receiving.RespOffCodes',
               'Hammerhead.store.receiving.StoreLoc',
               'Hammerhead.store.receiving.Conditions',
               'Hammerhead.store.receiving.Bins',
               'Hammerhead.grid.receiving.AssetItemGrid',
               'Hammerhead.grid.receiving.AssetReceivedItemGrid',
               'Hammerhead.grid.receiving.BoxQuantityGrid',
               'Hammerhead.store.receiving.Discrepancies'
               ],
    boxNumbersStore: Ext.create('Ext.data.ArrayStore', {
        fields: ['id'],
        data: []
    }),
    hideEditButtons: true,
    initialMaxBoxes: 0,
    addRowExpander:true,
    rowBodyTplX:[
                 '<div class="x-grid3-summary-row">',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Line # :</b> {line_number}</span>',
                 '<b>Tran Type :</b> {transactiontype}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Line Type :</b> {linetype}</span>',
                 '<b>Entered By :</b> {enteredby}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Item/Description :</b> {description}</span>',
                 '<b>Received Date :</b> {receiveddate}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Extended Description :</b> {extendeddescription}</span></div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Pack Slip # :</b> {pack_nbr}</span>',
                 '<b>Comments :</b> {comments}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Model :</b> {modelnum}</span>',
                 '<b>Manufacturer :</b> {manufacturer}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Part # :</b> {partnum}</span>',
                 '<b>Est Time of Arrival :</b> {venddeliverydate}</div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>Stock # :</b> {nsn}</span></div>',
                 '<div style="padding: 5px;"><span style="display: inline-block;width:
 350px;"><b>PO Line Comment :</b> {polinecomment}</span></div>',
                 '</div>'
             ],
    initComponent: function() {
        var me = this;
        uom = Hammerhead.util.Lookups.getLookupStore('unitOfMeasure', false);
        roc = Hammerhead.util.getStore('Hammerhead.store.receiving.RespOffCodes', fals
e, true, true);
        storeloc = Hammerhead.util.getStore('Hammerhead.store.receiving.StoreLoc', fal
se, true, true);
        rlds = Hammerhead.util.Lookups.getLookupStore('receivingDiscrepancy', false);
        conditions = Hammerhead.util.getStore('Hammerhead.store.receiving.Conditions',
```

```
 false, true, true);
        bins = Hammerhead.util.getStore('Hammerhead.store.receiving.Bins', false, true
, true);
        uomActive = Hammerhead.util.Lookups.getLookupStore('unitOfMeasure', true);
        me.rld = me.getFilteredDiscrepancyReasonStore();
        //me.bulkDiscrepancyImageId = me.createBulkDiscrepancyImageId();

//        if (!me.received) {
                me.selModel = Ext.create('Ext.selection.CheckboxModel', {
                        checkOnly : true,
                        allowDeselect : true,
                        mode : 'MULTI'
                });
//        }
            me.setNumBoxDropdownMax(me.initialMaxBoxes);
        me.columns = [{
                header: 'Line #',
                dataIndex: 'line_number',
                stateId: 'line_number',
                width: 60
        },{
            header: 'CLIN',
            dataIndex: 'clin',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            flex: 1,
            stateId: 'clin'
        },{
            header: 'Part #',
            dataIndex: 'partnum',
            width: 200,
            stateId: 'partnum',
            hidden: !Hammerhead.CurrentUser.isLOCUser() || me.received
        },{
                header: 'Description',
            dataIndex: 'description',
            width: Hammerhead.CurrentUser.isLOCUser() && !me.received ? 400 : 150,
            stateId: 'description',
            editor: {
                xtype: 'textfield',
                flex: 1,
                readOnly: true
//              disabled: true See Hammer-5400
                }
        },{
            header: 'Ord Qty',
            dataIndex: 'ord_qty',
            stateId: 'ord_qty',
            flex: 1
        },{
            header: 'Ord Unit',
            dataIndex: 'ord_uom',
            stateId: 'ord_uom',
            flex: 1
        },{
            header: 'Rec Qty',
            dataIndex: 'rec_qty',
            stateId: 'rec_qty',
            flex: 1,
            editor: {
                xtype: 'numberfield',
                allowBlank: true,
                minValue: 0,
                received: me.received,
                flex: 1,
                exisitingDiscrepant: me.exisitingDiscrepant,
```

```
                validator: me.recQtyValidator
            }
        },{
            header: 'Qty Due',
            dataIndex: 'qty_due',
            flex: 1,
            stateId: 'qty_due'
        },{
                header: 'Total Qty Rec',
            dataIndex: 'total_qty_received',
            flex: 1,
            stateId: 'total_qty_received'
        },{
            header: 'Rec Unit',
            dataIndex: 'rec_uom',
            stateId: 'rec_uom',
            flex: 1,
            editor: {
                xtype: 'lookupcombo',
                emptyText: 'Select a Unit of Measure',
                store: uomActive,
                allowBlank: false,
                flex: 1,
                listConfig: {minWidth: 200}
            },
            renderer: function(v) {
                var id = Ext.isObject(v) ? v.id : v;
                return id ? uom.getById(parseInt(id,10)).get('shortDescription') : '';
                //return id ? '' : '';
            }

        },{
            header: 'Pack Slip #',
            dataIndex: 'pack_nbr',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'pack_nbr',
            flex: 1,
            editor: {
                xtype: 'cleartextfield',
                flex: 1
            },
            renderer: function(v) {
                return /^\s+$/.test(v) ? '' : v;
            }
        },{
            //text: '<img id=' + me.bulkDiscrepancyImageId + ' style="vertical-align:
top;" alt="Update All Discrepancies" height=16 width=16 src="resources/images/discrepa
nt.gif" />' + ' Discrepancy',
            header:' Discrepancy',
            dataIndex: 'disc_cargo',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'disc_cargo',
            flex: 1,
            editor: {
                xtype: 'lookupcombo',
                listConfig: {minWidth: 200},
                flex: 1,
                store: me.rld
            },
            renderer: function(v) {
                var id = Ext.isObject(v) ? v.id : v;
                return id ? rlds.getById(parseInt(id,10)).get('shortDescription') : ''
;
            }
        },{
```

```
            header: 'Comments',
            dataIndex: 'comments',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'comments',
            flex: 1,
            align: 'center',
            editor: {
                xtype: 'cleartextfield'
            },
            renderer: function(v) {
                return /^\s+$/.test(v) ? '' : v;
            }
        },{
            header: 'Resp Off Code',
            dataIndex: 'respoffcode',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'respoffcode',
            flex: 1,
            editor: {
                xtype: 'combo',
                displayField: 'description',
                valueField: 'respoffcode',
                store: roc,
                flex: 1,
                listConfig: {minWidth: 200},
                queryMode: 'local'
            },
            renderer: function(v) {
                var id = Ext.isObject(v) ? v.value : v;
                return id ? roc.findRecord('respoffcode', id).get('description') : '';
            }
        },{
            header: 'Storeroom',
            dataIndex: 'storeloc',
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            flex: 1,
            stateId: 'storeloc'
        },{
            header: 'Rotating',
            dataIndex: 'rotating',
            stateId: 'rotating',
            flex: 1
        },{
                header: 'Condition',
                dataIndex: 'conditioncode',
                hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'conditioncode',
            flex: 1,
                editor: {
                        xtype: 'combo',
                        displayField: 'description',
                        valueField: 'value',
                        store: conditions,
                        flex: 1,
                        listConfig: {minWidth: 200},
                        queryMode: 'local'
                },
                renderer: function(v) {
                        var id = Ext.isObject(v) ? v.value : v;
                        return id != null && id != " " ? conditions.findRecord('value'
, id).get('description') : '';
                }
        },{
                header: 'Lot',
            dataIndex: 'lotnum',
```

```
            hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'lotnum',
            flex: 1,
            editor: {
                xtype: 'cleartextfield',
                maxLength:20,
                enforceMaxLength:true
            }
        },{
                header: 'Bin',
                dataIndex: 'binnum',
                hidden: Hammerhead.CurrentUser.isLOCUser() && !me.received,
            stateId: 'binnum',
            flex: 1,
                editor: {
                        xtype: 'combo',
                        displayField: 'binnum',
                        valueField: 'binnum',
                maxLength:50,
                minChars: 6,
                enforceMaxLength:true,
                        store: bins,
                        flex: 1,
                        listConfig: {minWidth: 200},
                        queryMode: 'local',
                        lastQuery: ''
                }
        },{
                xtype : 'actioncolumn',
            menuText:'Asset',
            stateId: 'asset',
            maxWidth:'40px',
            minWidth:'30px',
            sortable : false,
            flex: 1,
            hideable:false,
                icon : './resources/images/Add.png',
                tooltip : 'Click to open',
                text : 'Asset',
                handler : function(grid, rowIndex, colIndex) {
                        if (grid.getStore().getAt(rowIndex).get('receiptscomplete')) {
                                Ext.Msg.alert('Line Item Alert', 'This line item canno
t be edited. The Genesis PO Line is not in ORDERED status.');
                        } else {
                                boxId: me.boxid !== null ? me.boxid+1 : 1
                            var item = grid.getStore().getAt(rowIndex),
                            disc = item.data.disc_cargo !== null ? item.data.disc_carg
o : 0;
                                me.showAssetWindow(item.get('assetItems'), rowIndex,it
em.data.rec_qty,item.data.line_id, disc);
                        }
                }
        },{
            xtype : 'actioncolumn',
            menuText:'Box/Piece',
            stateId: 'boxPiece',
            hideable:true,
            hidden: !Hammerhead.CurrentUser.isAtWarehouseLocation(),
            maxWidth:'40px',
            minWidth:'30px',
            sortable : false,
            flex: 1,
            tooltip : 'Click to open',
            text : 'Box/Piece',
            getClass: function(v, meta, rec) {
```

```
                if (rec.data.boxItems.length > 0) {
                        return 'Check';
                } else {
                        return 'Add';
                }
            },
            handler : function(grid, rowIndex, colIndex) {
                if (grid.getStore().getAt(rowIndex).get('rec_qty') === null) {
                        Ext.Msg.alert('Error', 'Please enter a received quantity befor
e adding boxes/pieces to the line');
                } else if (me.getNumBoxesCombo() === null ||
                                me.getNumBoxesCombo().getValue() === null ||
                                me.getNumBoxesCombo().getValue() < 1 ) {
                                Ext.Msg.alert('Error', 'Please enter the number of Pie
ces before adding boxes/pieces to the line');
                } else {
                    me.showBoxWindow(grid.getStore().getAt(rowIndex).get('boxItems'),
rowIndex,
                                grid.getStore().getAt(rowIndex).get('rec_qty'),grid.ge
tStore().getAt(rowIndex).get('line_number'));
                }
            }
        },{
            dataIndex: 'syscomments',
            stateId: 'syscomments',
            flex: 1,
            hidden: true
        }
        ];
        me.extraButtons = [{
            text: me.received ? 'Return Balance Selected' : 'Receive Balance Selected'
,
            handler: me.received ? me.returnAll : me.receiveAll,
            scope: me,
            disabled: true
        },{
            text: 'Clear',
            handler: me.reset,
            scope: me,
            disabled: true
        }];
        /*
        me.addEvents('update');
        */
        me.callParent(arguments);
        /*
        me.mon(me.getView(), 'expandbody', me.fixSb, me);
        me.mon(me.getView(), 'collapsebody', me.fixSb, me);
        */
        //me.on('boxready', me.applyBulkDiscrepancyConfiguration, me);
        me.mon(me, 'render', function(obj, opts) {
                var numBoxesField = me.getNumBoxesCombo(), routeCombo = me.getRouteCom
bo();
                if (numBoxesField && numBoxesField.is('numberfield') && routeCombo &&
routeCombo.is('combobox')) {
                        me.mon(numBoxesField, 'change', me.onNumPiecesChange, me);
                        me.mon(routeCombo, 'change', me.onRouteChange, me);
                }
                me.setDisabled(me.checkDisableGrid());
       }, me);
        /* HAMMER-5246
        if (me.received)
        me.setDisabled(me.selModel);
        */
    }/*,
```

```
    applyBulkDiscrepancyConfiguration: function() {
        var me = this;

        var bulkDiscrepancyIcon = Ext.get(me.bulkDiscrepancyImageId);
        bulkDiscrepancyIcon.on("click", me.handleBulkDiscrepancyIconClick, me);

        Ext.create('Ext.tip.ToolTip', {
            target: bulkDiscrepancyIcon,
            html: 'Click to apply discrepancy to all line items'
        });

        bulkDiscrepancyIcon.addClsOnOver('hh-receiving-bulkdiscrepancy-mouseover-type'
);
    },
    handleBulkDiscrepancyIconClick: function(evt, el, o) {
        var me = this;

        var win = Ext.create('Hammerhead.window.receiving.ApplyBulkDiscrepancy', {
                listeners: {
                        applyBulkDiscrepancy: me.applyBulkDiscrepancy,
                        clearAllDiscrepancy: me.clearDiscrepancies,
                        scope: me
                }
        });
        win.show(this);
    },
    applyBulkDiscrepancy: function(discrepancyId) {
        var me = this;
        //HAMMER-3800 moved code to separate function for re-use.
        //var m = [];

        Ext.Array.each(me.getStore().getRange(), function(r) {
                r.set('disc_cargo', discrepancyId);
                me.setReceivedUnitOfMeasure(r);
        });

        //HAMMER-3800 moved code to separate function for re-use.
        me.updateLineItemLabel(discrepancyId);
    }*/,
    hasAnyValidLineItem: function() {
        var me = this;
        var recs = me.getReceivingLines();

        return recs.length > 0;
    },
    hasBoxPieces: function() {
        var me = this, rv = true;
        var recs = me.getReceivingLines();
                recs.each(function(r) {
                        if ((r.get('rec_qty') !== null && r.get('rec_qty') > 0) &&
                                (r.get('boxItems').length === 0) &&
                                (r.get('linetype') !== 'SERVICE' && r.get('linetype')
!== 'STDSERVICE') &&
                                Ext.isEmpty(r.get('disc_cargo'))) {
                                rv = false;
                        }
                });

        return rv;
    },
    hasReceivedLines: function() {
        var me = this, rv = true, lineCount = 0;
        var recs = me.getReceivingLines();
                recs.each(function(r) {
                        if ((r.get('rec_qty') !== null && r.get('rec_qty') > 0) &&
```

```
                                (r.get('linetype') !== 'SERVICE' && r.get('linetype')
!== 'STDSERVICE') &&
                                Ext.isEmpty(r.get('disc_cargo'))) {
                                lineCount++;
                        }
                });
        if (lineCount === 0) {
                rv = false;
        }

        return rv;
    },
    createBoxItem: function() {
        var me = this, b, ba = [],recs = me.getReceivingLines(), bi = me.boxid !== nul
l ? me.boxid : 1;
                recs.each(function(r) {
                        b = Ext.create('Hammerhead.model.receiving.BoxItem', {
                                r_lid: r.get('line_id'),
                                boxnum: ("00" + bi).slice(-3),
                                quantity: r.get('rec_qty')
                        });
                r.set('boxItems', Array(b.data));
                });
    },
    clearDiscrepancies: function() {
        var me = this;
        //me.applyBulkDiscrepancy();
        //HAMMER-3800 clear the discrepancy column, but only if its not null. Speed up
 the update
        //by not setting the Units and by suppressing events during update.
        me.getStore().suspendEvents();
        Ext.Array.each(me.getStore().getRange(), function(r) {
                if(!Ext.isEmpty(r.data.disc_cargo)){
                        r.set('disc_cargo', null);
                }
        });
        me.getStore().resumeEvents();
        me.getView().refresh();

        me.updateLineItemLabel();
    },
    //HAMMER-3800 added funciton
    updateLineItemLabel: function(discrepancyId) {
        var me = this;
        var m = [];

        //Update the 'Receiving Information' section with list of items received, if q
ty> 0 or item is discrepant.
                var recs = me.getReceivingLines();
                recs.each(function(r) {
                        m.push(r.get('line_number'));
                });
        m.sort(function(a,b) {return a-b;});
        me.fireEvent('update', me, m, Ext.isNumber(discrepancyId) ? true : false);
    },
    getFilteredDiscrepancyReasonStore: function() {
        var me = this;
        var fullStore = Hammerhead.util.Lookups.getLookupStore('receivingDiscrepancy',
 true, true);
        fullStore.clearFilter();
        fullStore.filter('discrepancyType',"LIVE");
        if (me.notCpsReceiving()) {
                me.filterSubstituteItemDiscrepancyFrom(fullStore);
        }
```

```
        return fullStore;
    },
    notCpsReceiving: function() {
        var me = this;
        return !me.cpsReceiving;
    },
    filterSubstituteItemDiscrepancyFrom: function(store) {
        var me = this;
        store.filter(me.recordNotSubstituteItem);
    },
    recordNotSubstituteItem: function(rec) {
        return rec.get('id') != Hammerhead.constants.RECEIVING_DISCREPANCY.SUBSTITUTE;
    },
    fixSb: function() {
        var me = this;
        /*
        me.determineScrollbars();       // TTD deprecated 4.1
        me.invalidateScroller();        // TTD deprecated 4.1
        */
    },
    reset: function() {
        var me = this, sm = me.getSelectionModel(), rec = sm.getSelection(),
            m = [], recs, v;
        Ext.Array.each(rec, function(r) {
            r.set('rec_qty');
            r.set('rec_uom',null);
            r.set('pack_nbr',null);
            r.set('disc_cargo',null);
            r.set('comments',null);
            r.set('respoffcode',null);
            r.set('binnum',null);
            r.set('lotnum',null);
            r.set('conditioncode',null);
            sm.deselect(r,true);
        });
        me.getDockedComponent(1).getComponent(3).setDisabled(true);
        me.getDockedComponent(1).getComponent(4).setDisabled(true);
        recs = me.getReceivingLines();
        recs.each(function(r) {
            m.push(r.get('line_number'));
        });
        m.sort(function(a,b) {return a-b;});
        me.fireEvent('update', me, m, v, false);
    },
    onSelectionChange: function(sm, recs) {
        var me = this;
        me.checkClear(recs);
    },
    checkClear: function(recs) {
        var me = this;
        if(!recs) {
            recs = me.getSelectionModel().getSelection();
        }
        me.getDockedComponent(1).getComponent(3).setDisabled(recs.length < 1); // || m
e.received);
        me.getDockedComponent(1).getComponent(4).setDisabled(recs.length < 1 || Ext.is
Empty(recs[0].data.rec_qty) || me.received);
    },
    recQtyValidator: function(v) {
        if (this.received) {
                if (this.exisitingDiscrepant && v < 1) {
                        return 'Received Quantity must be greater than 0';
                }
                if (! (v >= 0)) {
```

```
                        return 'Received Quantity cannot be blank';
                }
        }
        else {
                if (!(v > 0 || Ext.isEmpty(v))) {
                        return 'Received Quantity must be greater than 0';
                }
        }

        return true;
    },
    onBeforeEdit: function(rowediting, e) {
        var me = this;
        var ex = this.exisitingDiscrepant;
        var canEdit = !(me.received && e.record.get('rec_qty') == null) || ex;
        var i,col,ed, colBin, edBin;
        if (e.record.get('receiptscomplete')) {
                Ext.Msg.alert('Line Item Alert', 'This line item cannot be edited. The
 Genesis PO Line is not in ORDERED status.');
                return false;
        }
        if(canEdit) {
            // filter bin store
            if (e.field === 'binnum') {
                colBin = e.grid.columns[18];
                edBin = colBin.getEditor();
                edBin.store.clearFilter();
                edBin.store.filterBy(function(record) {
                        if (record.get('location') === e.record.get('storeloc')) retur
n record;
                });
            }

            for(i = 9; i<=11; i++) {
                col = e.grid.columns[i];
                ed = !col.hidden ? col.getEditor() : null;
                if(ed) {
                        // fyi below is commented out in prod
                        // TTD whats being checked for column 11
                    //ed.setDisabled((me.received && e.record.get(col.dataIndex) === n
ull) || (!me.received && i === 11) );
                    //
                    ed.setDisabled((me.received && e.record.get(col.dataIndex) === nul
l)  );
                }
            }
            me.setReceivedUnitOfMeasure(e.record);
        }
        return canEdit;
    },
    setReceivedUnitOfMeasure: function(record) {
        var me = this;

      if(!record.get('rec_uom')) {
          //var recUomId = Ext.getStore('UnitOfMeasures').findRecord('shortDescription
', record.get('ord_uom')).getId();
          var recUomId = uom.findRecord('shortDescription', record.get('ord_uom')).get
('id');
          record.set('rec_uom',recUomId);
      }
    },
    onCancelEdit: Ext.emptyFn,
    onEdit: function(rowediting,e) {
        var me = this,m=[],v,qc=0, qd=0,isDiscrepant=false,
        tqr = e.record.get('total_qty_received') !== null ?  e.record.get('total_qty_r
```

```
eceived') : 0,
        oq = e.record.get('ord_qty') !== null ?  e.record.get('ord_qty') : 0,
        orq = e.record.get('orig_rec_qty') !== null ?  e.record.get('orig_rec_qty') :
0,
        rq = e.record.get('rec_qty') !== null ?  e.record.get('rec_qty') : 0;

        if (e.record.get('rl_id') > 0 && e.record.get('orig_rec_qty') >= 0 && e.record
.get('rec_qty') === null) {
                e.record.set('rec_qty',0);
        }
        if (e.record.get('rl_id') == null && e.field === 'rec_qty') {
                qd = oq - tqr - rq + orq;
                if (qd < 0) {
                        qd = 0;
                }
                e.record.set('qty_due',qd);
        }

        var recs = me.getReceivingLines();
        if(recs.length !== 0) {
            recs.each(function(r) {
                m.push(r.get('line_number'));
            });
            m.sort(function(a,b) {return a-b;});
            if (e.record.get('genesis_id') > 0 && (e.record.get('rec_qty') > e.record.
get('orig_rec_qty')) ) {
                Ext.Msg.alert('Warning', 'Received quantity must be less than the orig
inal quantity received.');
                e.record.set('rec_qty',e.record.get('orig_rec_qty'));
                return false;
            }
            // handle early rotating returns
            if (e.record.get('genesis_id') > 0 && e.record.get('rotating') === 'Yes' )
 {
                qc = e.record.get('orig_rec_qty') - e.record.get('rec_qty');
                if (qc > 0 && e.record.get('assetReceivedItems').length == 0) {
                        /*Ext.Date.diff(new Date(),e.record.get('receiveddate'), "d")
*/
                    Ext.Msg.alert('Warning', 'You can\'t return this accountable item w
ithout specifying the assets you are returning. If Genesis has not yet created these a
ssets please allow Genesis an hour or two to process the new assets and then try again
. If that doesn't work contact Hammerhead support.');
                    e.record.set('rec_qty',e.record.get('orig_rec_qty'));
                    if (e.field === 'rec_qty') {
                        e.record.set('rec_qty',e.record.get('orig_rec_qty'));
                    }
                    return;
                }
            }
            // show respoffcodewindow if nulled out from grid line on returns
            if (e.record.get('genesis_id') > 0 && (e.record.get('respoffcode') === nul
l || e.record.get('respoffcode') === "") &&
               (e.record.get('rec_qty') < e.record.get('orig_rec_qty')) &&
               (e.field === 'rec_qty' || e.field === 'respoffcode')
                        ) {
                me.showRespOffCodeWindow(recs);
            }
            // check if return, rotating, and received quantity is lower
            if (e.record.get('genesis_id') > 0 && e.record.get('rotating') === 'Yes' )
 {
                qc = e.record.get('orig_rec_qty') - e.record.get('rec_qty');
                if (qc > 0 && e.record.get('assetReceivedItems').length > 0) {
                    me.showAssetReceivedWindow(e.record.get('assetReceivedItems'), e.re
cord.get('rl_id'), e.rowIdx, qc);
                }
```

```
            }
            if (me.cpsReceiving && e.field === 'rec_qty' && e.record.get('rotating') =
== 'Yes' && !(e.record.get('genesis_id') > 0)) {
                        disc = e.record.data.disc_cargo !== null ? e.record.data.disc_
cargo : 0;
                        me.showAssetWindow( e.record.data.assetItems, e.rowIdx,e.recor
d.data.rec_qty,e.record.data.line_id, disc);
            }
            if (e.record.get('disc_cargo') != null) {
                isDiscrepant = true;
                if( e.record.get('disc_cargo') === Hammerhead.constants.RECEIVING_DISC
REPANCY.SUBSTITUTE) {

                        disc = e.record.data.disc_cargo !== null ? e.record.data.disc_
cargo : 0;
                                me.showAssetWindow( e.record.data.assetItems, e.rowIdx
,e.record.data.rec_qty,e.record.data.line_id, disc);
                }
            }
            // check if overage or a return; loc only? per sean-everyone
            if (e.field === 'rec_qty' && !me.existingDiscrepant &&
                ((e.record.get('rl_id') == null && (rq + tqr) > oq) ||
                ((!me.draft && e.record.get('rl_id') != null) ||
                 (me.draft && e.record.get('rl_id') != null && rq > oq)))
                ) {
                me.showOverageWindow(e.record);
            };
            // check if rec qty changed
            if (e.record.get('rl_id') == null && e.field === 'rec_qty' &&
                e.originalValue != null &&
                e.originalValue != e.value &&
                e.record.get('boxItems').length != 0
                        ) {
                me.showBoxWindow(e.record.get('boxItems'), e.rowIdx, rq,e.record.get('
line_number'));
            }

            me.fireEvent('update', me, m, isDiscrepant);
            me.dirty = true;
        }
        else {
                me.fireEvent('update', me, [], false);
        }
        me.checkClear();
    },
    isPO: function() {
        var me = this, sm = me.getSelectionModel(), s = sm.getSelection(), ip = false;
        Ext.Array.each(s, function(r) {
                if (r.get('genesis_id') !== null && r.get('genesis_id') > 0) {
                        ip = true;
                }
        });
        return ip;
    },
    showReturnBalanceWindow: function() {
        var me = this,
                win = Ext.create('Hammerhead.window.InputWindow', {
            title: 'Quantity Change Comment',
            message: 'Please enter a comment that describes the reason for modifying t
he quantity on the line:',
            closable: false,
            includeCancel: false,
            listeners: {
                input: function(comment) {
                        // check if genesis
```

```
                        if (me.isPO()) {
                        me.showRespOffCodeForSelectedWindow(comment);

                        } else {
                        me.returnAllOk(comment);
                        }
                    }
                }
            });
            win.show(this);
        },
        showRespOffCodeForSelectedWindow: function(comment) {
            var me = this,
                    win = new Ext.Window({
                title: 'Select Responsible Office Code',
                layout: 'fit',
                width: 300,
                height: 300,
                closable: false,
                modal: true,
                items: [{
                    xtype: 'grid',
                    deferEmptyText: false,
                    emptyText: 'No responsible office codes found.',
                    loadingText: 'Loading Responsible Office Codes',
                    columns: [{
                        header: 'Responsible Office Codes',
                        dataIndex: 'description',
                        width: 300
                    }
                    ],
                    store: roc,
                    listeners: {
                        itemdblclick: function(v, record){
//                          recs.each(function(r) {
//                                  r.set('respoffcode', record.data.respoffcode);
//                            });
                            me.returnAllOk(comment, record.data.respoffcode);
                            win.close();
                        }
                    }
                }]
            });
            win.show(this);
        },
        receiveAll: function() {
            var me = this;
            Ext.Msg.prompt('Receive All Selected', 'Packing Slip # (optional):', me.receiv
eAllOk, me);
        },
        returnAll: function() {
            var me = this;
            me.showReturnBalanceWindow();
//          Ext.Msg.prompt('Return All Selected', 'Responsible Office Code (required):',
 me.returnAllOk, me);
        },
        receiveAllOk: function(btn, text) {
            if(btn === 'ok') {
                var me = this,
                    sm = me.getSelectionModel(),
                    s = sm.getSelection(),
                    m = [],
                    anyReceiptsComplete = Ext.Array.some(s, function(r) {
                            return r.get('receiptscomplete')
                    });
```

```
            if (anyReceiptsComplete) {
                Ext.Array.each(s, function(r) { sm.deselect(r, true) });
                Ext.Msg.alert('Line Item Alert', 'This line item cannot be edited. The
 Genesis PO Line is not in ORDERED status.');
            } else {
                Ext.suspendLayouts();
                Ext.Array.each(s, function(r) {
                        var total_qty_received = r.get('total_qty_received') != null ?
 r.get('total_qty_received') : 0,
                                orig_rec_qty = r.get('orig_rec_qty') != null ? r.get('
orig_rec_qty') : 0,
                                ord_qty = r.get('ord_qty') != null ? r.get('ord_qty')
: 0,
                                uom = Hammerhead.util.Lookups.getLookupStore('unitOfMe
asure', false);

                    m.push(r.get('line_number'));
                    r.set('rec_qty', ord_qty - orig_rec_qty - total_qty_received);
                    r.set('qty_due', ord_qty - r.get('rec_qty') - total_qty_received);
                    r.set('rec_uom', uom.findRecord('shortDescription', r.get('ord_uom
')).getId());
                    r.set('pack_nbr', text);
                    sm.deselect(r,true);
                });
                Ext.resumeLayouts(true);
                m.sort(function(a,b) {return a-b;});
                me.fireEvent('update', me, m, false);
            }

            me.getDockedComponent(1).getComponent(3).setDisabled(true);
            me.getDockedComponent(1).getComponent(4).setDisabled(true);
        }
    },
    returnAllOk: function(text, respoffcode) {
        var me = this, sm = me.getSelectionModel(), s = sm.getSelection(),
        m = [],
        total_qty_received, orig_rec_qty,ord_qty, isReceiptscomplete = false; ;
        Ext.suspendLayouts();
        Ext.Array.each(s, function(r) {
                total_qty_received = r.get('total_qty_received') != null ? r.get('tota
l_qty_received') : 0;
                orig_rec_qty = r.get('orig_rec_qty') != null ? r.get('orig_rec_qty') :
 0;
                ord_qty = r.get('ord_qty') != null ? r.get('ord_qty') : 0;
                if (r.get('receiptscomplete')) {
                        isReceiptscomplete = true;
                }
                if (!r.get('receiptscomplete') && orig_rec_qty > 0 && (r.get('rotating
') !== null && r.get('rotating') !== 'Yes') ) {
                m.push(r.get('line_number'));
                r.set('rec_qty', 0);
                r.set('syscomments', text);
                if (respoffcode !== null) {
                    r.set('respoffcode', respoffcode);
                }
                sm.deselect(r,true);
                }
        });
        Ext.resumeLayouts(true);
        m.sort(function(a,b) {return a-b;});
        if (isReceiptscomplete) {
                Ext.Msg.alert('Line Item Alert', 'Line items cannot be edited. The Gen
esis PO Line is not in ORDERED status.');
        }
```

```
        me.fireEvent('update', me, m, false);
        me.getDockedComponent(1).getComponent(3).setDisabled(true);
        me.getDockedComponent(1).getComponent(4).setDisabled(true);
    },
    showRespOffCodeWindow: function(recs) {
        var win = new Ext.Window({
            title: 'Select Responsible Office Code',
            layout: 'fit',
            width: 300,
            height: 300,
            closable: false,
            modal: true,
            items: [{
                xtype: 'grid',
                deferEmptyText: false,
                emptyText: 'No responsible office codes found.',
                loadingText: 'Loading Responsible Office Codes',
                columns: [{
                    header: 'Responsible Office Codes',
                    dataIndex: 'description',
                    width: 300
                }
                ],
                store: roc,
                listeners: {
                    itemdblclick: function(v, record){
                        recs.each(function(r) {
                                r.set('respoffcode', record.data.respoffcode);
                        });
                        win.close();
                    }
                }
            }]
        });
        win.show(this);
    },
    showAssetsReceivedWindow: function(rec) {
        var win = new Ext.Window({
            title: 'Select Assets to be returned',
            layout: 'fit',
            width: 300,
            height: 300,
            closable: false,
            modal: true,
            items: [{
                xtype: 'grid',
                deferEmptyText: false,
                emptyText: 'No assets found.',
                loadingText: 'Loading Assets',
                columns: [{
                    header: 'Asset Number',
                    dataIndex: 'assetnum',
                    width: 100
                },{
                    header: 'Description',
                    dataIndex: 'description',
                    width: 200
                }
                ],
                store: Ext.getStore('receiving.AssetsReceived'),
                listeners: {
                    itemdblclick: function(v, record){
                        rec.set('assetsreturned', record.data.assetsreturned);
                        win.close();
                    }
```

```
                }
            }]
        });
        win.show(this);
    },
    showOverageWindow: function(rec) {
        var overunderdesc = rec.get('rl_id') == null ? 'Overage' : 'Underage',
                win = Ext.create('Hammerhead.window.InputWindow', {
            title: 'Quantity Change Comment',
            message: 'Please enter a comment that describes the reason for modifying t
he quantity on the line:',
            initialInput:rec.data.syscomments,
            closable: false,
            includeCancel: false,
            listeners: {
                input: function(comment) {
                    rec.set('syscomments',comment);
                }
            }
        });
        win.show(this);
    },
    anyDiscrepanyLines: function() {
        var me = this;

        var recs = this.store.queryBy(function(r) {
            return !Ext.isEmpty(r.get('disc_cargo'));
        });

        return recs.length > 0;
    },
    getReceivingLines: function() {
        var recs = this.store.queryBy(function(r) {
            return Ext.valueFrom(r.get('rec_qty'), -1) >= 0 || (Hammerhead.CurrentUser
.isLOCUser() && !Ext.isEmpty(r.get('disc_cargo')));
        });
        return recs;
    },
    getValue: function() {
        var lis = [];
        this.getReceivingLines().each(function(r) {
            lis.push(r.data);
        });
        return lis;
    },
    onNumPiecesChange: function(piecesField, newVal, oldVal, eOpts) {
        this.setNumBoxDropdownMax(newVal);
        this.setDisabled(this.checkDisableGrid());
    },
    setNumBoxDropdownMax: function(newMax) {
        var me = this;
        var currentMax = me.boxNumbersStore.getCount();
        if (newMax > currentMax) {
                for (var i = currentMax + 1 ; i <= newMax ; i++) {
                        me.boxNumbersStore.add({id: i});
                }
        } else if  (newMax < currentMax) {
                me.boxNumbersStore.remove(me.boxNumbersStore.data.getRange(newMax, cur
rentMax - 1));
        }
    },
    getParentForm: function() {
        return this.up('receivingform');
    },
    getNumBoxesCombo: function() {
```

```
        return this.getParentForm().down('[name="pieces"]');
    },
    getRouteCombo: function() {
        return this.getParentForm().down('[name="route"]');
    },
    onRouteChange: function(routeField, newVal, oldVal, eOpts) {
        this.setDisabled(this.checkDisableGrid());
    },
    checkDisableGrid: function() {
        return  false;
    },
    checkDisableGridOld: function() {
        return  this.getParentForm().editable &&
                        Hammerhead.CurrentUser.isLOCUser() && (
                                this.getNumBoxesCombo() === null ||
                                this.getNumBoxesCombo().getValue() === null ||
                                this.getNumBoxesCombo().getValue() < 1 ||
                                this.getRouteCombo() === null ||
                                this.getRouteCombo().getValue() === null ||
                                this.getRouteCombo().getValue() === ''
                        )
    },
    showAssetWindow: function(rec, rowIndex,recQty,line_id,disc) {
        var me = this, rotating = me.store.data.items[rowIndex].data.rotating;
        if (Ext.isEmpty(recQty) || recQty == 0){
            Ext.Msg.alert('Rec Qty Missing', 'Must have at least 1 qty available befor
e being able to add an asset.');
            return;
        }
        var win = Ext.create('Hammerhead.window.receiving.AssetItems', {
            title: 'Assets',
            rec: rec,
            rowIndex: rowIndex,
            line_id: line_id,
            rotating: rotating,
            recQty:recQty,
            disc:disc,
            received: me.received,
            cpsReceiving:me.cpsReceiving
        });

        win.on('submit', me.refreshAssetItems, me);
        win.show(this);
    },
    refreshAssetItems: function(params) {
        var me = this;
        me.store.getAt(params.rowIndex).set('assetItems',params.assetItems);
    },
    showBoxWindow: function(rec, rowIndex, qty, lineNumber) {
        var me = this,
        win = Ext.create('Hammerhead.window.receiving.BoxItems', {
            title: 'Box/Piece Allocation For Line Item ' + lineNumber,
            rec: rec,
            rowIndex: rowIndex,
            boxId: me.boxid !== null ? me.boxid : 0,
            rec_qty: qty,
            pieces: me.getNumBoxesCombo(),
            line_id: me.store.data.items[rowIndex].data.line_id,
            received: me.received,
            existingDiscrepant: me.existingDiscrepant
        });
        win.on('submit', me.refreshBoxItems, me);
        win.show(this);
    },
    refreshBoxItems: function(params) {
```

```
        var me = this;
//        me.boxid = params.boxId;
        me.store.getAt(params.rowIndex).set('boxItems',params.boxItems);
    },
    showAssetReceivedWindow: function(rec, rl_id, rowIdx, qc) {
        var me = this,
        win = Ext.create('Hammerhead.window.receiving.AssetReceivedItems', {
            title: 'Assets Received',
            rec: rec,
            rowIndex: rowIdx,
            line_id: rl_id,
            qty_change: qc
        });
        win.on('submit', me.refreshAssetReceivedItems, me);
        win.show(this);
    },
    refreshAssetReceivedItems: function(params) {
        var me = this;
        me.store.getAt(params.rowIndex).set('assetReceivedItems',params.assetReceivedI
tems);
    },
    getErrors: function() {
        var me = this, errors = [];

        if(!me.hasAnyValidLineItem() && !me.isReceivingOverride){
                errors.push(me.invalidReceivingValidationMsg);
        }

        return errors;
    },
    isValid : function() {
        var me = this;

        return me.hasAnyValidLineItem() || me.isReceivingOverride;
    }
    // Cannot use itemId on image tag as is, so using a salt to get unique ID for this
 users browser in case they have
    // multiple tabs open in their browser.
    /*createBulkDiscrepancyImageId: function() {
        var me = this;
        return 'discrepancyIcon' + me.getSalt();
    },
    getSalt: function() {
        return Ext.Number.randomInt(1,1000);
    }*/
});
```